D D M M Y Y Y Y

Module - 02

01) Explain array of pointers to objects and accessing member functions using pointers.

→ Array of pointers to objects

- A pointer is a variable that stores the memory address of another variable as it's value.
- Pointers to objects are to make a pointer that can access the object, not the variables. Pointer to object in c++ refers to accessing an object.
- There are two approaches by which you can access an object. One is directly & another one is using pointer to an object in c++.
- A pointer to an object in c++ is used to store the address of the object.

Syntax: ClassName *pointerobject;

- For storing the address of an object into a pointer in c++ we use the syntax:

pointerobject = &objectname;

Ex:

```
#include<iostream>
using namespace std;
class My_class
{
    int num;
public:
    void set_number(int value)
    { num = value; }
    void show_number();
};
```

```
void My_class :: show_number()
{
cout << num << "\n";
}
int main()
{
My_class object, *p;
object.set_number(1);
object.show_number();
p = &object;
p -> show_number();
return 0;
}
```

Accessing member functions using pointers

→ When you have a pointer to an object, there are two ways to access it's members.

i) Using the → operator

```
ptr->functionName();
```

ii) Using the *and.operators (less common)

```
(*ptr).functionName();
```

Ex:

```
Student *S = new student;
S->setData(101, "Ravi");
S ->display();
(*S).setData(101, "Ravi");
(*S). display();
```

D D M M Y Y Y Y

02) Explain the terms with an example.

→ i) pointers to class members

> We can use pointer to point to class's data members

Syntax for declaration:

datatype class_name :: *pointer_name;

Syntax for Assignment:

pointer_name = &class_name :: datamember_name;

Both declaration & assignment can be done in a single statement too.

datatype class_name :: *pointer_name =
&class_name :: datamember_name;

Using pointers with objects

> For accessing normal data members we use the dot. operator with object and -> with pointer to object.

object.*pointertoMember

objectpointer -> pointertoMember

Ex

```
class Data {
 public: int a;
      void print()
     {
     cout << "a is " << a;
     };
int main()
{
 Data d, *dp;
 dp = &d;
 int Data :: *ptr = &Data :: a;
 d.*ptr = 10;
```

```
d.print();
dp->*ptr=20;
dp->print();
```

o/p

```
a is 10
a is 20
```

ii) The this pointer

→ The "this pointer" holds the address of current object, in simple words you can say that this pointer points to the current object of the class.

Syntax:

```
void functionName()
{
this->memberName=value;
}
```

Ex:

```
#include<iostream>
using namespace std;
class DEMO
{
private: int num;
    char ch;
public:
  void setMyvalues(int num,char ch)
    {
  this->num=num;
  this->ch=ch;
    }
void displayMyvalues()
```

```
{
cout << num << endl;
cout << ch;
}
};
int main()
{
Demo obj;
obj.setMyvalues (100, 'A');
obj.displayMyvalues ();
return 0;
}
```

o/p

100
A

03) Write c++ program to demonstrate function overloading for the following prototypes. i) Add (int a, int b) ii) Add (double a, double b)

→

```
#include<iostream>
#include <conio.h>
int add(int a, int b) {
return a+b;
}
double add ( double a, double b) {
return a+b;
}
void main () {
clrscr ();
int x, y;
```

```
double p, q;
cout << "Enter two integers:";
cin >> x >> y;
cout << "Enter two decimal numbers:";
cin >> p >> q;
cout << "\n sum of integers = " << add(x,y);
cout << "\n sum of doubles = " << add(p,q);
getch();
}
```

o/p

```
Enter two integers : 10  5
sum of integers = 15
Enter two decimal numbers : 1.0  1.5
sum of doubles = 2.5
```

04) Write various function overloading used to "default function arguments" and "ambiguity".

--> A default argument is a value provided in a function declaration that is automatically assigned by the compiler if the calling function doesn't provide a value for the argument.

Ex:

```
#include<iostream>
using namespace std;
void display(char='*', int=3);
int main()
{
int count=5;
count << "NO argument passed:";
display();
```

```
cout << "First argument passed: ";
display ('#');
cout << "Both arguments passed: ";
display ('$', count);
return 0;
}
void display (char c, int count)
{
for (int i=1; i<=count; ++i)
{
cout<<c;
}
cout <<endl;
}
```

O/P

No argument passed: ***
First argument passed: ###
Both arguments passed: $$$$$

Ambiguity

→ Ambiguity arises when the compiler cannot decide which function to call because more than one function matches the call equally well.

Ex:

```
#include<iostream>
using namespace std;
void test (int a) {
cout << "Integer version" <<endl;
}
void test (double a) {
```

D D M M Y Y Y Y

```
cout << "Double version" << endl;
}
int main() {
  test ('A');
  return 0;
}
```

Q5) What is the need of constructor? How is it different from the member function?

→ Constructor: A constructor is a special member function of a class that is automatically called when an object is created.

Syntax

```
class className {
  public:
  className() {
    //
  }};
```

Need of constructor

i) Automatic Initialization: They initialize objects automatically when they are created, saving you from calling seperate function

ii) Ensures object is Ready to use: Before any operation, a constructor ensures all data members are in a valid state.

iii) Code Reusability: You can create multiple objects with different initial values using parameterized constructors.

Difference b/w constructor and Member function

Feature	Constructor	Member Function
Purpose	Used for initializing objects	used for performing operations on data member.
Name	same as class name.	Any valid function name.

D D M M Y Y Y Y

Return type	No return type(not even void)	Must have a return type
Invocation	Called automatically when object is created.	Must be called explicitly by the user.
overloading	Can be overload.	Can also be overloaded.
Inheritance	Constructors are not inherited.	Member functions are inherited.
Default call	Automatically executed once per object.	Executed when explicitly called.

06) What is i) copy constructor ii) Dynamic constructor explain with ex.

→ i) copy constructor

→ copy constructor are the member functions of a class that initialize the data members of the class using another object of the same class.

Syntax

```
class class_name {
class_name (class_name & old_object) {
var_name = old_object.var_name;
----
----
} }
```

Ex:

```
#include<iostream>
using namespace std;
class student {
  string name;
  int age;
public:
student (string n, int a) {
    name = n;
```

```
    age = a;
}
Student (const Student &S) {
    name = S.name;
    age = S.age;
    Cout << "Copy constructor called!" << endl;
}
void display () {
Cout << "Name:" << name << ", Age:" << age << endl;
};
int main () {
Student S1 ("Ravi", 20);
Student S2 = S1;
    S2.display ();
    return 0;
}
```

O/P

Copy constructor called!
Name: Ravi, Age: 20.

ii) Dynamic constructor

→ It is a constructor that allocates memory dynamically use the new operator when an object is created.

→ It is useful when:

- The size of data is not known at compile time.
- You want to allocate memory at runtimes.

Ex:

```
#include <iostream>
#include <cstring>
using namespace std;
```

```
class string {
char *name;
int length;
public:
string (const char *s) {
   length = strlen (s);
name = new char [length +1];
 strcpy (name, s);
cout << "Dynamic constructor called! << endl;
 void desplay () {
 cout << "Name: " << name << endl;
 }
  string () {
   delete [] name;
cout << "Memory released!" << endl;
};
int main() {
string s1 ("C++ Programming");
 s1. desplay ();
 return 0;
}
```

o/p

Dynamic constructor called!
Name: C++ programming.
Memory realased!